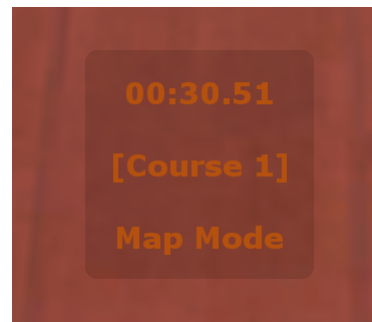


Problem A. Cheater Detection

Source file name: Cheater.c, Cheater.cpp, Cheater.java, Cheater.py
 Input: Standard
 Output: Standard

The old classic computer game, Twisted Forest 2, has recently become really popular to speedrun. Fans of the game have made an unofficial leaderboard for who can beat the game the fastest. The times are given in seconds, with 2 digits of precision, as reported by the in-game timer. You suspect that there are cheated times on the leaderboard. In order to prove this, you've decompiled the game and taken a close look at the source code for the in-game timer:

```
time = 0.0
for each tick:
    displayed_time = floor(100 * time)/100
    display(displayed_time)
    time += 0.015
```



You've realized that the displayed time can skip over some numbers. For example it is impossible for the timer to output 5.39 since the closest possible times are 5.385 (displayed as 5.38) and 5.400 (displayed as 5.40). Maybe you can use this insight to catch some cheaters!

Warning: Floating point numbers are imprecise and can give unexpected results. However, you can **assume that no floating point errors occur in the in-game timer.**

Input

The first line contains N , the number of times to check ($1 \leq N \leq 10^5$).

The following N lines each contain a real number t_i ($0 \leq t_i \leq 10^6$). These numbers will always have exactly two digits after the decimal point.

Output

For each number t_i in the input, print "VALID" if it can be achieved in the game, or "IMPOSSIBLE" if it cannot be achieved.

Example

Input	Output
5	VALID
0.00	IMPOSSIBLE
0.02	IMPOSSIBLE
1.13	VALID
1.14	VALID
9.99	

Problem B. Blackthorn

Source file name: Blackthorn.c, Blackthorn.cpp, Blackthorn.java, Blackthorn.py
Input: Standard
Output: Standard

Blackthorn is a plant that grows in abundance in the fields around Stockholm. They form huge maze-like bushes that are home to many animals, and sometimes humans. This time of year (in early May), the blackthorn bushes are covered in beautiful white flowers. Go to the fields near Lill-Jansskogen after the contest and see for yourself!

The word blackthorn is also special because it contains `kth` as a substring. This means that the letters `kth` appears consecutively in the word. In this problem, you are given a word, and your task is to decide if it contains `kth` as a substring.



Input

The input consists of a string `s` of length between 3 and 100. This string only consists of lowercase letters “a”-“z”.

Output

If `s` contains `kth` as a substring, print “yes”. Otherwise, print “no”.

Example

Input	Output
blackthorn	yes
kitakyushu	no

Problem C. Crochet Competition

Source file name: Crochet.c, Crochet.cpp, Crochet.java, Crochet.py
Input: Standard
Output: Standard

The National Crochet Pattern Competition (NCPC) is an annual team-based event for crocheters of all skill levels, from beginner to expert. Teams design, write, and crochet original patterns within a time limit, which can range from minutes to a full week depending on the complexity—sometimes spanning weekends. Contestants may take breaks to eat, sleep, or go home, but the timer continues running.

NCPC has asked you to develop a system that calculates the total competition duration from the start and end times, provided with weekday, hour, and minute. The reported duration should be precise, correctly expressed in days, hours, and minutes, using as few components as necessary while minimizing the sum of the integer values.



Crocheting. License: CC BY-SA 3.0 by floret on Wikimedia Commons.

Input

Input consists of two lines.

- The first line gives the competition's start time.
- The second line gives the competition's end time.

Times are formatted as "**Www HH:MM**", where

- **Www** is the three-letter weekday abbreviation: **Mon, Tue, Wed, Thu, Fri, Sat, or Sun.**
- **HH** the two-digit hour, from 00 to 23.
- **MM** is the two-digits minutes, from 00 to 59.

If the start and end times are identical, the competition lasts exactly one week.

Output

Output the competition duration in days, hours, and minutes, using the singular or plural form as appropriate (e.g., 1 day, 2 days). List the components in order – days, hours, minutes – separating three components with commas and two with "and"; omit any component with a value of zero.

Example

Input	Output
Mon 08:00 Mon 15:00	7 hours
Mon 10:00 Wed 08:59	1 day, 22 hours, 59 minutes
Fri 20:00 Mon 08:00	2 days and 12 hours

Problem D. Dune Dash

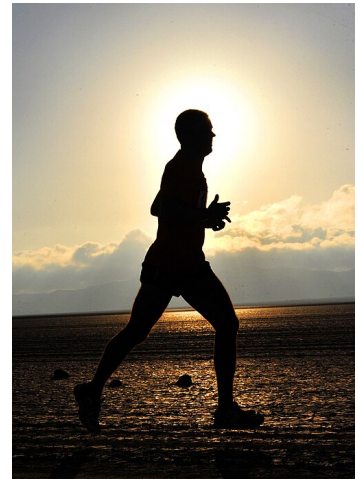
Source file name: Dune.c, Dune.cpp, Dune.java, Dune.py
Input: Standard
Output: Standard

You signed up for the *Dune Dash*, a running race across the desert. Everything went well — except that in the excitement, you forgot to start *StrideTrack*, the app that records how far you've run. All you have now are the official checkpoint locations, but not the order in which you passed through them.

Formally, the race consisted of N checkpoints, each given by its coordinates in the Euclidean plane. The sequence in which they were visited is unknown to you, but the organizers designed the course to prevent anyone from straying off route. In particular, if $q_1, q_2, \dots, q_N$ is the correctly ordered list of the checkpoints along the race, then for every triple $i < j < k$ it holds that

$$\text{dist}(q_i, q_k) > \max(\text{dist}(q_i, q_j), \text{dist}(q_j, q_k))$$

where $\text{dist}(p, q)$ denotes the Euclidean distance between points p and q . Your task is to determine the total length of the race.



A runner during a 15-kilometer run through the Grand Bara Desert in Djibouti. Public domain photo by Joshua Bruns.

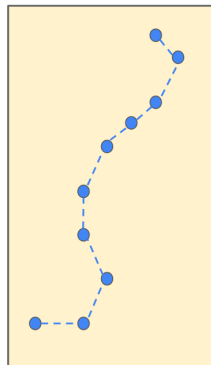


Figure 1. Illustration of example 2. The dashed line shows where the race went.

Input

The first line consists of the integer N ($2 \leq N \leq 2 \cdot 10^5$). The following N lines each contain two integers x_i and y_i ($-10^9 \leq x_i, y_i \leq 10^9$). These are the coordinates of each checkpoint.

The checkpoints are not necessarily in the order in which they were visited during the race. It is guaranteed that there is some ordering of the checkpoints such that they satisfy the distance requirements above.

The N points given in the input are all distinct.

Output

Print one floating point number, the length of the race. Your answer will be correct if it has an absolute or relative error of at most 10^{-6} .



Example

Input	Output
3 1 0 0 0 1 1	2.0
10 -1 -7 -1 -11 0 -9 2 2 1 -2 2 -1 3 1 -1 -5 0 -3 -3 -11	17.186912597118443

Problem E. Dice Distributions

Source file name: Dice.c, Dice.cpp, Dice.java, Dice.py
 Input: Standard
 Output: Standard

Mikael and Harry are planning to play a game called *Maessiw* against each other. The rules of *Maessiw* are extremely complicated, so we will not describe them here.

To play *Maessiw*, each player needs two dice. The two dice used by a single player are not required to have the same number of sides. However, the probability distribution of the sum of those two dice must be identical for both players.

We say that two probability distributions are identical if they assign the same probability to every possible outcome.

We model an N -sided die as a discrete uniform distribution over its faces: each face is labeled with an integer (labels need not be distinct or ordered), and each face occurs with probability $1/N$.

Currently, Mikael already has two dice: one with A sides and one with B sides. Harry has only one die with C sides. To play the game, Harry needs a second die such that the distribution of the sum of his two dice is identical to the distribution of the sum of Mikael's dice.

Additionally, Harry would like his new die to have as few sides as possible, to make the die and the game as simple as possible.

Can you help Harry design his second die?

Input

The first line of input contains the integers A, B, C , ($1 \leq A, B, C \leq 2 \cdot 10^5$), where A and B are the number of sides of Mikael's two dice, and C is the number of Harry's first die.

The second line contains A integers a_i ($1 \leq a_i \leq 2 \cdot 10^5$), the number written on the i -th side of Mikael's first die.

The third line contains B integers b_i ($1 \leq b_i \leq 2 \cdot 10^5$), the number written on the i -th side of Mikael's second die.

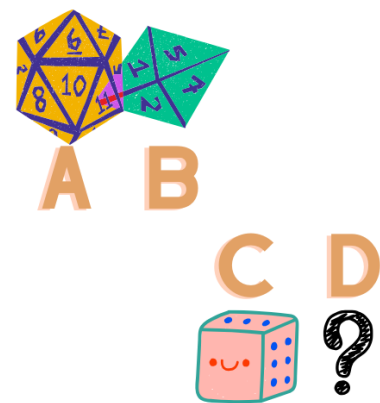
The fourth line contains C integers c_i ($1 \leq c_i \leq 2 \cdot 10^5$), the number written on the i -th side of Harry's first die.

Output

First print an integer D on a single line, the minimum number of sides that Harry's second die could have. On the second line, print D integers, $d_1, d_2, \dots, d_D$ representing the numbers written on each of the sides in non-decreasing order.

Note that the numbers d_i can exceed $2 \cdot 10^5$.

It is guaranteed that a solution exists for the given input, and it can be proven that the answer is unique. It is also guaranteed that $D \leq 2 \cdot 10^5$.



Sprites made by @sparklestroke on Canva.

Example

Input	Output
6 6 6 1 2 3 4 5 6 1 2 3 4 5 6 1 2 3 4 5 6	6 1 2 3 4 5 6
9 9 9 1 1 1 1 2 2 2 2 3 5 4 4 3 3 3 2 2 1 1 1 2 3 2 3 2 3 4	9 1 1 2 2 2 3 3 3 4
9 9 18 1 1 2 3 2 3 2 3 4 1 1 2 2 2 3 3 3 4 1 1 1 1 2 2 2 2 3 1 1 1 1 2 2 2 2 3	9 1 2 2 3 3 3 4 4 5
3 4 6 4 4 5 4 4 4 4 3 3 3 3 4 4	1 5

Explanation

In example 4, Mikael has dice with 3 sides and 4 sides, and Harry's first die has 6 sides. The smallest die possible that can be given to Harry such that Harry's and Mikael's probability distributions are identical, is a die with 1 side (Do NOT question how physically to create a die with a single side).

We can verify that the probability distributions of their sums are identical.

Mikael has the following 12 outcomes, resulting in a probability of $\frac{8}{12} = \frac{2}{3}$ for obtaining 8, and $\frac{4}{12} = \frac{1}{3}$ for obtaining 9.

	$b_1 = 4$	$b_2 = 4$	$b_3 = 4$	$b_4 = 4$
$a_1 = 4$	8	8	8	8
$a_2 = 4$	8	8	8	8
$a_3 = 5$	9	9	9	9

Harry has the following 6 outcomes, resulting in a probability of $\frac{4}{6} = \frac{2}{3}$ for obtaining 8, and $\frac{2}{6} = \frac{1}{3}$ for obtaining 9.

	$d_1 = 5$
$c_1 = 3$	8
$c_2 = 3$	8
$c_3 = 3$	8
$c_4 = 3$	8
$c_5 = 4$	9
$c_6 = 4$	9

Thus, Mikael and Harry have identical probability distributions.

Problem F. Dream Luck

Source file name: Dream.c, Dream.cpp, Dream.java, Dream.py
Input: Standard
Output: Standard

The next big craze on the video sharing site TockTick is the “Mega Luck Challenge”. The videos are daily gaining views in the millions. This week the most popular videos were about a soda machine accidentally giving two drinks instead of one, finding a 1000 SEK bill on the ground or narrowly dodging a falling tree branch. Your plan is to take part in this Mega Luck Challenge. So you have recorded yourself rolling a $(10^9 + 1)$ -sided die N -times in a row to show off your mega luck. Unfortunately it turned out that you were not really lucky, and the result from the recording was not impressive to say the least.

Your back up plan is to instead upload a shorter clip from the long recording where you happen to be lucky. You define the Dream Luck of a clip as the frequency of the most frequent outcome. For example if you are able to find a clip where two thirds of the rolls result in a 6, then that would have a dream luck of $2/3$.

You’ve decided that you want to upload a clip containing at least K rolls with the maximum possible dream luck. Given a list of outcomes of the N rolls as well as the integer K , find the interval of length $\geq K$ that maximizes the dream luck!



Thumbnail of "How lucky is too lucky?" on TockTick, video by Sit-down Codes

Input

The first line of input contains the integers N and K ($1 \leq K \leq N \leq 3 \cdot 10^5$).

The second line contains N integers a_i ($0 \leq a_i \leq 10^9$).

Output

Print one real number, the maximum dream luck among all intervals of length $\geq K$. Your answer will be correct if it has an absolute or relative error of at most 10^{-6} .

Example

Input	Output
3 2 1 2 1	0.666666666667
7 3 0 2 2 2 3 2 4	1
7 5 1 2 3 4 5 6 6	0.4

Problem G. KTH Challenge Posters

Source file name: Posters.c, Posters.cpp, Posters.java, Posters.py
Input: Standard
Output: Standard

KTH Challenge is back! For this year, Harry and Nils are tasked with hanging up as many posters as possible.

With Harry's incredible skills of using a single printer to print out posters, and Nils' incredible skills to hang up posters, they team up and want to put up as many posters as possible within T seconds.

It takes Harry P seconds to print out each poster. After a poster has been printed, Nils can immediately take it and hang it up in H seconds.

We assume that the printer has an infinite supply of paper, and that Harry and Nils can each work independently at the same constant speed without disturbing one another. Additionally, the printer can only print one poster at a time, and Nils can hang up only one poster at a time.

Given that they have T seconds before they have to go home for the day, what is the maximum amount of posters they can put up?

Input

The input consists of a single line with the three integers P , H , and T ($1 \leq P, H, T \leq 10^9$), which represent the values mentioned in the statement.

Output

Print an integer: the maximum amount of posters Harry and Nils can hang up within T seconds.

Example

Input	Output
3 5 8	1
4 4 17	3
5 3 14	2



KTH Challenge 2025 poster.

Problem H. Heavy Edges

Source file name: Heavy.c, Heavy.cpp, Heavy.java, Heavy.py
Input: Standard
Output: Standard

Björn went to the local grocery store, the T-Fast, and came back carrying an extremely heavy... graph?

In a graph, an edge e is said to be a *heavy edge* if there exists a simple cycle that contains e in which e is the heaviest edge (equal or heavier weight than all other edges in the cycle).

Björn would like to know how heavy the graph he brought back is. Given a connected, undirected, weighted graph, find the number of heavy edges.



The T-Fast near KTH. (2014)

Input

The input will describe the graph that Björn brought back.

The first line of input contains the two integers N and M ($1 \leq N, M \leq 5 \cdot 10^5$), the number of vertices and edges in the graph.

The following M lines each contain three integers u_i, v_i, w_i ($1 \leq u_i, v_i \leq N$, $1 \leq w_i \leq M$), meaning that an edge goes between vertices u_i and v_i of weight w_i .

The graph will always be connected, and contains no self-loops or multiple edges. This implies that $u_i \neq v_i$ for all $i = 1, \dots, M$, and there is at most one edge between each pair of nodes.

Output

Print an integer: the number of heavy edges in the graph.



Example

Input	Output
5 7 1 2 7 2 3 6 3 1 5 3 4 4 4 2 3 2 5 2 5 4 1	3
5 5 1 2 3 2 3 1 3 4 4 4 5 1 5 1 5	1
5 6 1 2 100 2 3 100 3 1 100 3 4 1 4 5 1 5 1 1	3
5 4 1 2 99 2 3 99 3 4 99 4 5 99	0

Problem I. Instagraph

Source file name: Instagraph.c, Instagraph.cpp, Instagraph.java, Instagraph.py
 Input: Standard
 Output: Standard

A celebrity in a social network is somebody with many followers, but who doesn't follow them back. More precisely, a person is a *celebrity* for a group of people, if

- every member of the group follows the person,
- the person follows nobody in the group.

The *celebrity centrality* of person v , written $CC(v)$, is the maximum size of such a group.

We model the social network as a directed graph with N vertices $1, \dots, N$. A directed edge from u to v means that person u follows person v . For example, in

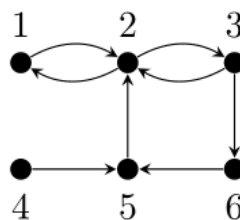


Figure 2. Example Input 1

we have $CC(1) = 0$, $CC(2) = 1$, and $CC(5) = 2$.

Your task is to find a vertex v with the maximum celebrity centrality $CC(v)$. In case of a tie, choose the smallest v .

Input

The input consists of

- One line with two integers N and M ($1 \leq N \leq 200\,000$, $0 \leq M \leq 1\,000\,000$), the number of vertices and the number of directed edges.
- M lines with two distinct integers u and v ($1 \leq u, v \leq N$), indicating a directed edge from u to v . There are no duplicate edges.

Output

Output two integers: the smallest v with the maximum celebrity centrality and the value $CC(v)$.





Example

Input	Output
6 8 1 2 2 1 2 3 3 2 3 6 4 5 5 2 6 5	5 2
1 0	1 0

Problem J. Jump

Source file name: Jump.c, Jump.cpp, Jump.java, Jump.py
Input: Standard
Output: Standard

The Unsightly Halfwits are putting on a wild concert in their natural habitat of Húsavík. Their lead singer, Sveinn “Springheel” Jónsson, is leaping about the stage with vigorous energy, turning every note into a spectacle. But keeping in rhythm while bouncing so chaotically is no easy task, and you can’t help but wonder how long he can keep this up.

Sveinn starts with an energy level of E . The song plays a beat exactly every N milliseconds. His first jump launches at time 0, perfectly in sync with the music. As long as his energy E is strictly positive ($E > 0$), each jump lasts exactly E milliseconds. When his energy runs out with $E = 0$, he collapses and the show is over. If he still has energy after landing, he immediately springs up again for the next jump. When Sveinn lands exactly on a beat, he gains +1 energy from sheer enjoyment. But if he’s even slightly off-beat, his energy drops by 1.

Input

The input consists of:

- One line with two integers N, E , the number of milliseconds between beats and the initial energy level of the lead singer. These will satisfy $1 \leq N \leq 3 \cdot 10^5$ and $1 \leq E \leq 10^9$.

Output

Output a single integer, the number of milliseconds that will pass before Sveinn stops jumping. If he never stops, instead print **infinity**.

Example

Input	Output
7 9	59
7 4	infinity
100001 123123123	7581167638793024

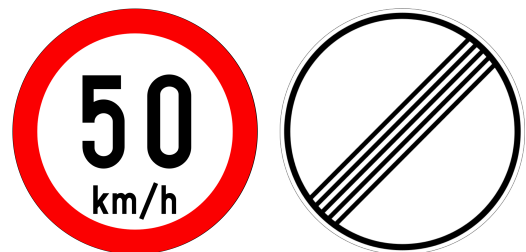


Ballsout at Metal Frenzy Open Air 2022, image by S. Bollmann, commons.wikimedia.org

Problem K. Km/h

Source file name: Kmh.c, Kmh.cpp, Kmh.java, Kmh.py
Input: Standard
Output: Standard

Many countries have a “national speed limit”: a default maximum speed that applies unless a *speed limit sign* explicitly sets a lower one. Such a speed limit sign shows a number and overrides the default while it is in force. By contrast, a *derestriction sign* shows no number at all; it cancels the previous restriction and restores the national speed limit, which is always strictly higher than any speed limit sign.



A speed limit sign (left) and a derestriction sign (right). National variants differ, but are regulated by the *Vienna Convention on Road Traffic* (1968).

You pride yourself on being a careful and attentive motorist. Whenever you encounter a speed limit sign, you smoothly and safely adjust your vehicle’s speed. Alas, despite your otherwise impeccable memory, the derestriction sign always leaves you guessing. You simply cannot remember the national speed limit – only that it was a neat multiple of ten.

As you encounter each sign – limit or derestriction – determine the fastest you can go while abiding the law, based only on what you’ve seen so far.

Input

The input consists of:

- One line with an integer N with $1 \leq N \leq 100$, the number of signs.
- N lines, one for each sign. Each line describes either a speed limit or a derestriction. A speed limit is an integer L with $10 \leq L \leq 160$, the speed limit that applies from that sign. A derestriction is the forward slash “/”, reminiscent of the sign’s diagonal bar. The first line contains a speed limit. There are never two consecutive derestrictions.

Output

Output N integers, the maximum legal speed that can be inferred when passing the i th sign for $1 \leq i \leq N$.

Example

Input	Output
8	90
90	85
85	100
/	119
119	120
/	85
85	85
85	120
/	

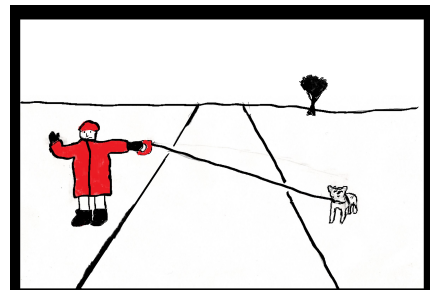
Problem L. Road Block

Source file name: Road.c, Road.cpp, Road.java, Road.py
 Input: Standard
 Output: Standard

The town Bikelona is most famous for containing the headquarters of the big company BIKEA. As you might expect, every employee at BIKEA goes to and from work by bicycle. Since the employees are obsessed with efficiency, they always take a shortest path from where they live to the office.

Cattis and her dog also live in Bikelona. They think that it is a shame that the BIKEA employees are so stressed out and always have to take the shortest path to work. So Cattis decides to block one road in town, so that some people will have to take a longer route. That way, the cyclists will have more time to reflect and find inner peace.

Bikelona can be represented as an undirected graph with N vertices and M edges. The BIKEA headquarters are situated in vertex 1. There are x_v BIKEA employees living in vertex v , for every v . Your task is to find, for each edge in the graph, how many people are forced to take a longer path from their home to vertex 1, if that edge is removed. Not being able to get to work at all also counts as having to take a longer path.



Input

The first line of input contains the two integers N and M ($2 \leq N \leq 3 \cdot 10^5$, $1 \leq M \leq 3 \cdot 10^5$).

The second line of input contains the N integers x_v ($0 \leq x_v \leq 10^9$). x_1 will always be 0, because no one lives and works in the same place.

The following M lines each contain two integers u_i, v_i ($1 \leq u_i, v_i \leq N$), meaning that an edge goes between vertices u_i and v_i .

The graph will always be connected, and contains no self-loops or multiple edges.

Output

Print M lines with one integer y_e on each, the number of people who are forced to take a longer route if the edge e is removed. The edges are numbered from 1 to M in the order in which they appear in the input.



Example

Input	Output
4 4 0 10 3 7 1 2 2 3 2 4 3 4	20 3 7 0
5 6 0 1 2 100 100 1 2 1 3 2 4 2 5 3 4 3 5	1 2 0 0 0 0